



PES

UNIVERSITY

CELEBRATING 50 YEARS

ALGORITHMS ANALYSIS AND DESIGN

Lekha A

Computer Applications

ALGORITHMS ANALYSIS AND DESIGN

Introduction, Analysis Framework and Sorting Techniques Introduction

Lekha A

Computer Applications



ALGORITHMS ANALYSIS AND DESIGN

Algorithms

- Algorithms play a central role in science and practice of computing.
- The study of the algorithms is known as **algorithmics**.
- It is the core of computer science and is said to be relevant to most of science, business and technology.



ALGORITHMS ANALYSIS AND DESIGN

Algorithms

- An algorithm is a set of rules for carrying out calculation either by hand or on a machine.
- An algorithm is a finite step-by-step procedure to achieve a required result.
- An algorithm is a sequence of computational steps that transform the input into the output.



ALGORITHMS ANALYSIS AND DESIGN

Algorithms

- An algorithm is a sequence of operations performed on data that have to be organized in data structures.
- An algorithm is an abstraction of a program to be executed on a physical machine (model of Computation).
- An algorithm is a finite sequence of instructions, an explicit, step-by-step procedure for solving a problem.



ALGORITHMS ANALYSIS AND DESIGN

Algorithms

- Algorithm is a term used to define the steps needed to create a standard solution to a standard problem.
- It is a sequence of unambiguous instructions for solving a problem i.e. for obtaining a required output for any legitimate input in a finite amount of time.



ALGORITHMS ANALYSIS AND DESIGN

Algorithms

- The name comes after Persian mathematician “*Mohammed Ibn Musa Alkhowariznie*”.

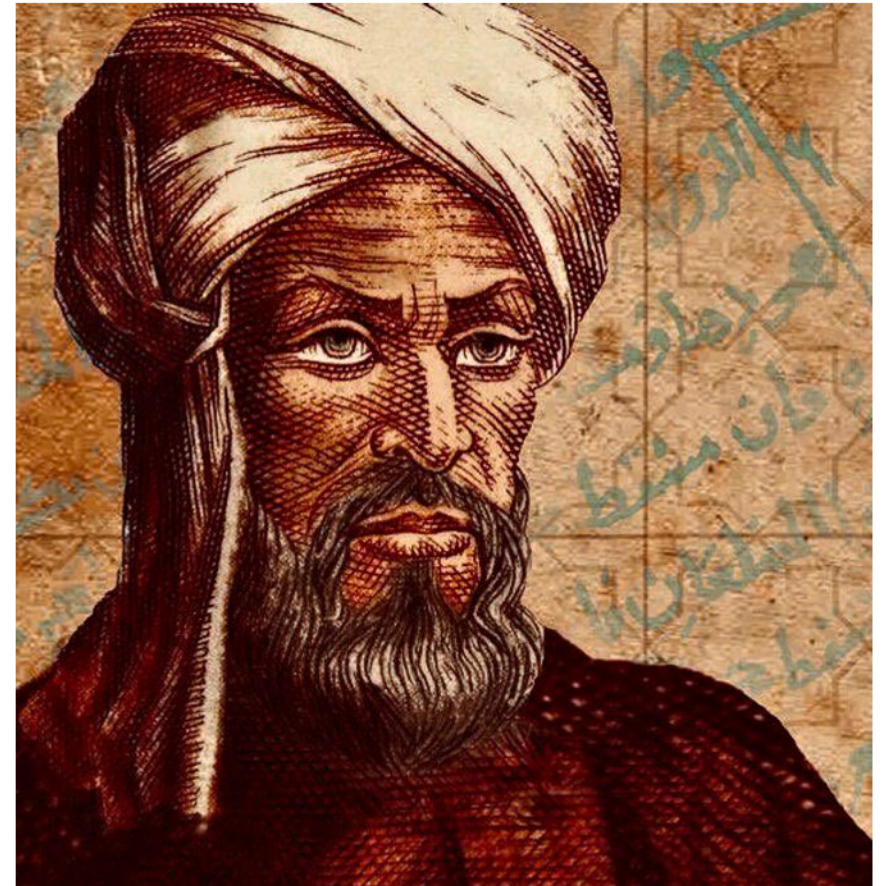


Fig 1 – Adapted from <https://www.baytalfann.com/post/algorithms-algebra-astronomy-muhammad-ibn-musa-al-khwarizmi>



ALGORITHMS ANALYSIS AND DESIGN

Earliest Algorithms

- Euclid's algorithm, which was developed 2,000 years ago, is utilized for determining the greatest common divisor of two numbers.
- The Urdhva Tiryakbhyam algorithm, employed by Vedic mathematicians, facilitates rapid integer multiplication.



ALGORITHMS ANALYSIS AND DESIGN

Earliest Algorithms

- Arithmetic algorithms, such as a division algorithm, were utilized by the Babylonians and Egyptians.
- The Sieve of Eratosthenes, a method used by Greek mathematicians, assists in identifying prime numbers.



ALGORITHMS ANALYSIS AND DESIGN

Earliest Algorithms

- Cryptographic algorithms, which were developed by Arabic mathematicians, are employed for code-breaking purposes.
- The Chinese Remainder Theorem, which was discovered by Chinese mathematicians in the 3rd century AD, is significant in number theory.
- The Babylonian-Sumerian technique for extracting roots represents one of the earliest documented instances of a mathematical algorithm.



ALGORITHMS ANALYSIS AND DESIGN

Earliest Algorithms

- Gauss's Gaussian elimination algorithm is a method for solving systems of linear equations.
- The "Chakravala" method, a cyclic algorithm created by Brahmagupta, Bhaskara, and Jayadeva, is used to solve indeterminate quadratic equations.



ALGORITHMS ANALYSIS AND DESIGN

Earliest Algorithms

- Gauss's Gaussian elimination algorithm is a method for solving systems of linear equations.
- The "Chakravala" method, a cyclic algorithm created by Brahmagupta, Bhaskara, and Jayadeva, is used to solve indeterminate quadratic equations.



ALGORITHMS ANALYSIS AND DESIGN

Properties of Algorithms

- An algorithm has the following five basic properties
 1. A number of quantities are provided to an algorithm initially before the algorithm begins. These are the **inputs** that are processed by the algorithm.
 2. The processing rules specified in the algorithm must be **precise**, **unambiguous** and lead to a specific action. An instruction which can be carried out is called an **effective instruction**.



ALGORITHMS ANALYSIS AND DESIGN

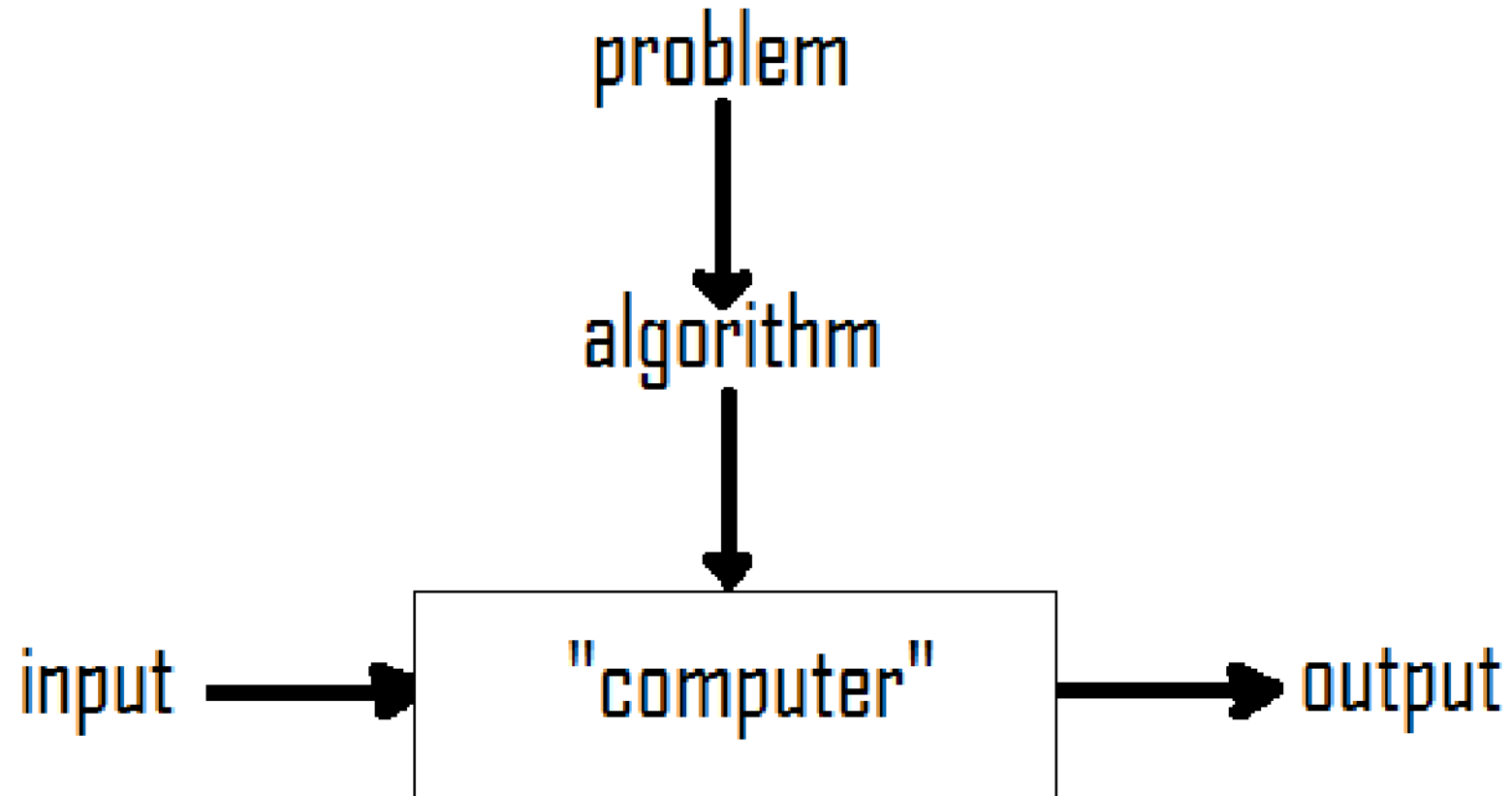
Properties of Algorithms

- An algorithm has the following five basic properties
 3. Each instruction must be sufficiently **basic** such that it can be carried out in a **finite time** by a person with paper and pencil.
 4. The **total time** to carry out all the steps in the algorithm must be **finite**.
 5. An algorithm must have one or more **output**.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm Process





ALGORITHMS ANALYSIS AND DESIGN

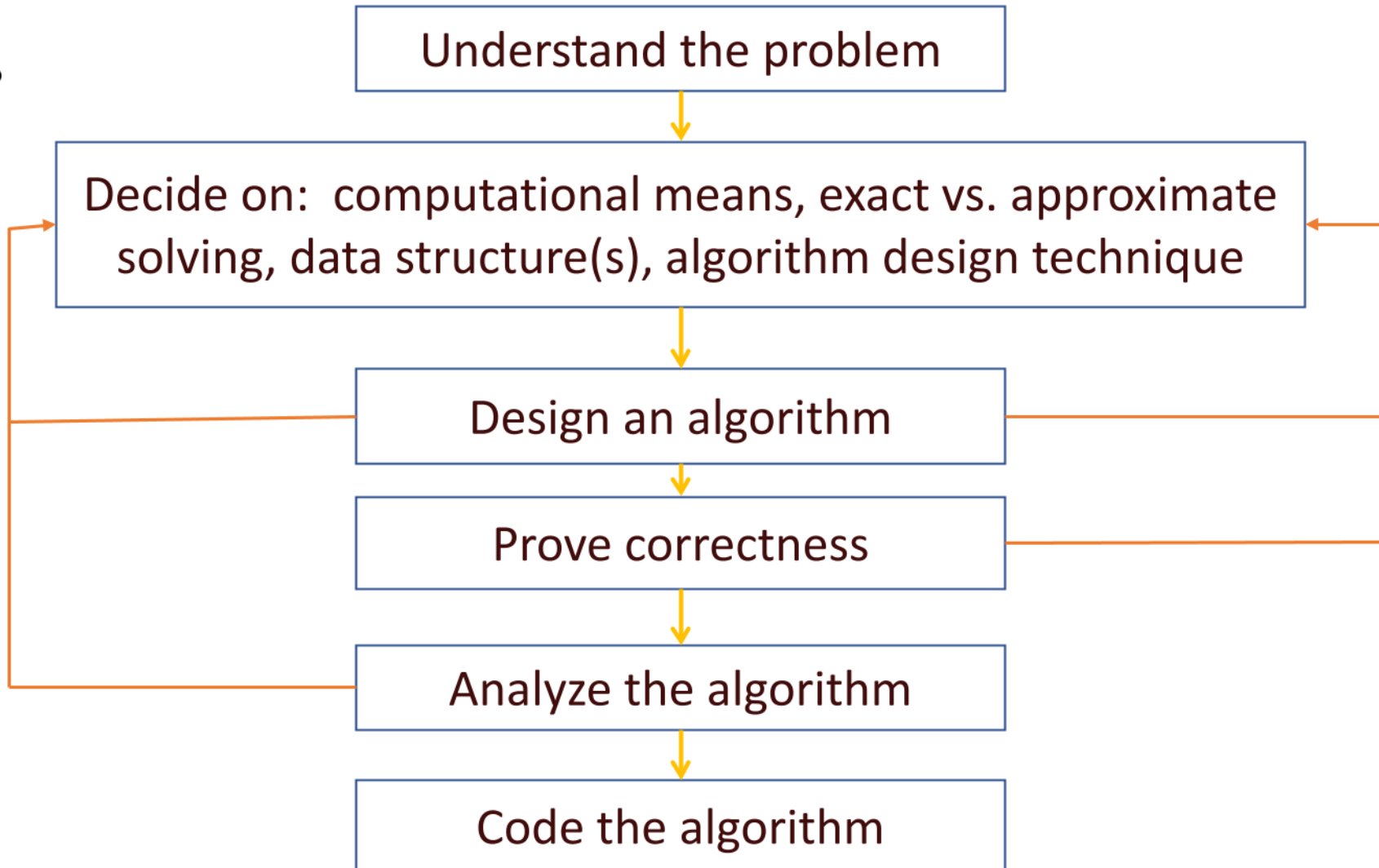
Fundamentals of Algorithmic Problem Solving

- Algorithms are considered to be procedural solutions to problems.
- The solutions are not answers to the problems but specific instructions for getting the answers.



ALGORITHMS ANALYSIS AND DESIGN

Fundamentals of Algorithmic Problem Solving





ALGORITHMS ANALYSIS AND DESIGN

Understand the Problem

- Understand the problem completely.
- Have to know if a known algorithm exists for solving the problem.
- If it exists use it.
- Understand how the algorithm works, know its strengths and weaknesses.



ALGORITHMS ANALYSIS AND DESIGN

Understand the Problem

- Input to an algorithm represents an instance of the problem the algorithm solves.
- Specify exactly what is the range of the instances the algorithm needs.
 - If this is not specified the algorithm may work for a majority of values but may crash at the boundary values.
- A correct algorithm works correctly for all legitimate inputs.



ALGORITHMS ANALYSIS AND DESIGN

Ascertaining the capabilities of a computational device

- Many of the algorithms that are in use now are destined to work for a computer that closely resembles the von Neumann machine.
- The architecture's main concept is the Random Access Machine.
- The assumption is that instructions are executed one after another, one operation at a time.
 - They are called **sequential algorithms**.



ALGORITHMS ANALYSIS AND DESIGN

Ascertaining the capabilities of a computational device

- Some newer computer can execute operations in parallel.
- The algorithms that take advantage of this capability are called **parallel algorithms**.



ALGORITHMS ANALYSIS AND DESIGN

Choosing between the exact and approximate problem solving

- Problem can be solved exactly or approximately.
- Sometimes it is necessary to opt for approximation algorithm because,
 - There are some problems which can not be solved exactly like finding square roots.



ALGORITHMS ANALYSIS AND DESIGN

Choosing between the exact and approximate problem solving

- Exact algorithms available may be unacceptably slow because of the problem's intrinsic property.
- Approximation algorithm may be a part of a sophisticated algorithm that solves a problem exactly.



ALGORITHMS ANALYSIS AND DESIGN

Deciding on appropriate data structures

- Some algorithms do not demand any ingenuity in representing their inputs.
 - Dynamic programming, Sorting
- Some of the algorithm designing techniques
 - Works on ingenious data structures
 - Depends intimately on structuring data, specifying a problem instance.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm Design Techniques

- It is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
- Reasons to study different techniques
 - They provide guidance for designing algorithms for new problems.
 - Problems that do not have satisfactory solutions.



ALGORITHMS ANALYSIS AND DESIGN

Algorithm Design Techniques

- Algorithm design technique makes it possible to classify algorithms according to an underlying idea.
- They serve as a natural way to categorize and study the algorithms



ALGORITHMS ANALYSIS AND DESIGN

Methods of specifying an algorithm

- Using Natural Language
 - This method leads to ambiguity.
 - Clear description of algorithm is difficult.
- Using Pseudo codes
 - It is a mixture of natural language and programming language like constructs.
 - It is more precise than a natural language.



ALGORITHMS ANALYSIS AND DESIGN

Proving an algorithm's correctness

- Prove that the algorithm yields required result for every legitimate input in a finite amount of time.
- Example – Euclid's algorithm
 - Euclid's algorithm is based on applying
 - $\text{gcd}(m,n) = \text{gcd}(n, m \bmod n)$



ALGORITHMS ANALYSIS AND DESIGN

Proving an algorithm's correctness

- Step 1: If $n == 0$ return the value of 'm' as the answer and stop; else proceed to Step 2.
- Step 2: Divide 'm' by 'n' and assign the value of the remainder to 'r'.
- Step 3: Assign the value of 'n' to 'm' and the value of 'r' to 'n'. Go to Step 1.



ALGORITHMS ANALYSIS AND DESIGN

Proving an algorithm's correctness

- Analysis
 - The second number gets smaller on each iteration of the algorithm
 - The algorithm stops when the second number becomes zero.



ALGORITHMS ANALYSIS AND DESIGN

Proving an algorithm's correctness

- For some algorithms, a proof of correctness is quite easy ; for others, it can be quite complex.
- Common technique
 - Use mathematical induction
 - An algorithm is incorrect if it fails for at least one instance of input.
 - If it is incorrect then it has to be redesigned.



ALGORITHMS ANALYSIS AND DESIGN

Analysing an Algorithm

- Two kinds of algorithm efficiency
 - Time efficiency
 - How fast the algorithm runs.
 - Space efficiency
 - How much extra space the algorithm needs.



ALGORITHMS ANALYSIS AND DESIGN

Analysing an Algorithm

- Other desirable characteristics
 - Simplicity
 - Simpler algorithms are easier to understand but depends on the user.
 - Generality
 - Two issues
 - Generality of the problem the algorithm solves.
 - Range of inputs.



ALGORITHMS ANALYSIS AND DESIGN

Analysing an Algorithm

- Other desirable characteristics
 - Simplicity
 - Simpler algorithms are easier to understand but depends on the user.
 - Generality
 - Two issues
 - Generality of the problem the algorithm solves.
 - Range of inputs.



ALGORITHMS ANALYSIS AND DESIGN

Generality of the problem the algorithm solves

- It is easier to design an algorithm for a problem proposed in more general terms.
- Example- Determine whether two integers are relatively prime.
 - Two integers are relatively prime if they share no common positive factors (divisors) except 1.
 - It is easier to do this with designing an algorithm to find the greatest common divisor and check whether the gcd is 1 or not.



ALGORITHMS ANALYSIS AND DESIGN

Generality of the problem the algorithm solves

- It does not mean that for every problem, a general algorithm will be designed.
- Example
 - Standard formula for roots of a quadratic equation cannot be generalized to handle polynomials of arbitrary degrees.



ALGORITHMS ANALYSIS AND DESIGN

Range of Inputs

- The algorithm should handle a range of inputs that is natural for the problem.
- Example
 - Not having 0 or 1 as one of the inputs to calculate gcd is unnatural.



ALGORITHMS ANALYSIS AND DESIGN

Coding an Algorithm

- Good algorithm is a result of repeated effort and rework
- Most of the algorithms are destined to be implemented on computer programs.
- Not only implementation but also the optimization of the code is necessary.
 - This increases the speed of operation.



ALGORITHMS ANALYSIS AND DESIGN

Coding an Algorithm

- Good algorithm is a result of repeated effort and rework
- Most of the algorithms are destined to be implemented on computer programs.
- Not only implementation but also the optimization of the code is necessary.
 - This increases the speed of operation.



ALGORITHMS ANALYSIS AND DESIGN

Recap

- What is an Algorithm?
- Fundamentals of Algorithmic Problem Solving



THANK YOU

Lekha A
Computer Applications